Escola Tècnica Superior d'Enginyeria de Telecomunicació de Barcelona
UNIVERSITAT POLITÈCNICA DE CATALUNYA

Gestió dinàmica de memòria

Programació i Estructures de Dades

Grau en Enginyeria Electrònica de Telecomunicació

ETSETB Telecom BCN

Introducció

- **L'ordinador reserva memòria automàticament per emmagatzemar-hi les variables que declarem**:
  - Si les declarem dins d'una funció, es reserva memòria a l'executar aquesta, que s'allibera en sortir
  - La quantitat de memòria que cal reservar es decideix en temps de compilació (tipus de dada, nombre de posicions del vector, etc...)
- Aquesta **reserva automàtica de memòria pot limitar-nos en certs casos**:
  - De quantes posicions hauríem de declarar un vector, quan no sabem a priori quants elements necessitarem desar-hi?
- El llenguatge C ens ofereix **funcions per reservar i alliberar memòria manualment durant l'execució del programa**: la que necessitem, però només durant el temps que la necessitem!

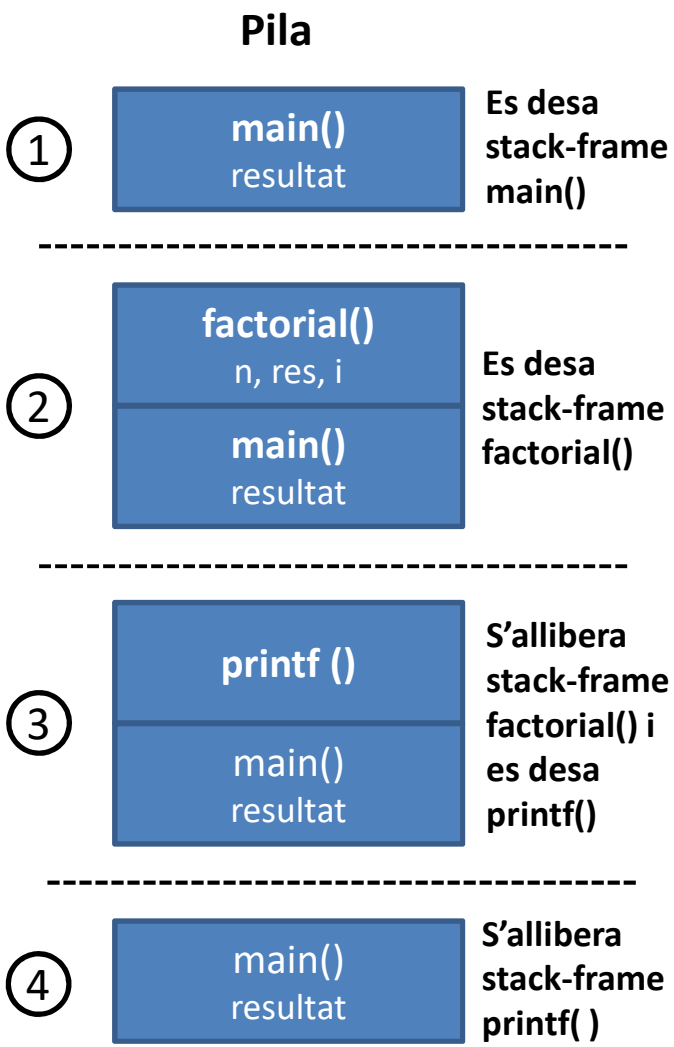
Tipus de memòria d'un programa C

- Un programa C emmagatzema totes les dades que gestiona **en tres àrees diferents de memòria** de l'ordinador:
  - **Memòria global/estàtica**: variables declarades fora de qualsevol funció, que s'emmagatzemen de principi a final del programa
  - **Pila**: variables locals de funcions, que només estan disponibles durant la seva execució. Continuament s'emmagatzemen i s'esborren variables en aquesta àrea, tot realitzat de forma automàtica
  - **Heap**: memòria que pot ser reservada i alliberada en qualsevol moment de l'execució del programa. Se n'encarrega el programador, mitjançant funcions de llibreria. **El heap també rep el nom de <u>memòria dinàmica</u>**

Funcionament de la pila

```
int factorial(int n) {
  int res = 1; int i;
  if(n < 0){
      return(-1);
  } else{
      for(i = n; i >= 2; i--){
         res *= i;
      }
      return(res);
  }
}

int main() {
   int resultat;          1
   resultat = factorial(4);   2
   printf("4! = %d\n", resultat);   3
   return(EXIT_SUCESS);   4
}
```



Utilització de la memòria dinàmica

- L'àrea de **memòria global/estàtica i la pila tenen una mida fixa**, obtinguda a partir del codi (variables declarades, etc...)
- El **heap pot abastar la totalitat de memòria de l'ordinador**, o aquella que el sistema operatiu permet adreçar a un procés individual
- El mecanisme d'ús de la memòria dinàmica consisteix en:
  - Reservar bloc de memòria necessari (funció *malloc* o *calloc*)
  - Redimensionar bloc prèviament reservat (funció *realloc*)
  - Alliberar bloc de memòria prèviament reservat quan ja no es necessiti (funció *free*) <- **MOLT IMPORTANT !!!**
- **Si no s'allibera la memòria dinàmica reservada**, aquesta quedarà inutilitzada permanentment: **Fuita de memòria (memory leak)**

Funció malloc() de stdlib.h

- La funció `void * malloc(size_t size);` s'encarrega de reservar un nombre consecutiu de Bytes igual al valor del seu paràmetre `size`. Retorna la @memòria inicial del bloc reservat, o NULL si la reserva no s'ha pogut efectuar correctament
- Normalment, ens interessarà emmagatzemar una variable d'un cert tipus (o diverses d'aquest tipus de forma consecutiva). Per obtenir quants Bytes ocupa en memòria un tipus de variable, podem usar l'operador `sizeof()`:

```
/* Reserva per guardar un enter i un nombre complex */
int *valor = (int *)malloc(sizeof(int));
t_complex *c = (t_complex *)malloc(sizeof(t_complex));

if (valor != NULL) /* Comprovem que malloc() OK */
    *valor = 16;
if (c != NULL) /* Comprovem que malloc() OK */
{
    c->real = 2.0; c->imaginària = 4.0;
}
```

Funció calloc() de stdlib.h

- La funció `void * calloc(size_t nmemb, size_t size);` és similar a `malloc()`. Reserva un bloc de memòria per guardar-hi `nmemb` elements de mida `size` Bytes consecutivament
- A diferència de `malloc()`, que no inicialitzava la memòria del bloc reservat, `calloc()` sí que inicialitza tots els Bytes reservats a 0
- Exemple: vector dinàmic de mida proporcionada en temps d'execució

```
int num_pos;
printf("Nombre de posicions del nou vector d'enters: ");
scanf("%d", &num_pos);
/* Reserva d'un bloc de memòria per al nou vector */
int *vector = (int *)calloc (num_pos, sizeof(int));
if (vector != NULL) {
/* Totes les posicions inicialment = 0 */
 /* ... */
}
```

Funció realloc() de stdlib.h

- La funció `void * realloc(void *ptr, size_t size);` es pot usar per redimensionar el bloc de memòria dinàmica prèviament reservat apuntat per `ptr`, de forma que ara tingui mida igual a `size` Bytes
- Retorna la @memòria inicial del bloc nou, que pot ser diferent de la del bloc original. Les dades del bloc de memòria antic que hi càpiguen al nou es mantenen al llarg del redimensionat
- Exemple: Redimensionat del vector dinàmic de la transparència anterior, multiplicant x2 la seva mida

```
int *tmp_ptr = (int *)realloc (vector, 2*num_pos*sizeof(int));
if (tmp_ptr != NULL) {
   /* Si redimensionat OK, apuntem nou bloc amb punter vector */
   vector = tmp_ptr;
}
```

Funció free() de stdlib.h

- La funció `void free(void *ptr);` allibera el bloc de memòria dinàmica prèviament reservat apuntat per `ptr`. Després de cridar a `free()`, tots els Bytes del bloc de memòria alliberat podran ser reservats novament
- Un cop finalitzat l'ús d'un bloc de memòria dinàmica l'hem d'alliberar mitjançant `free()`
- Alliberem tots els blocs de memòria reservats i establim el valor dels punters a NULL per tal evitar potencials problemes (p.e., sobreescriptura de dades a través d'un punter que continua apuntant a un bloc ja alliberat):

```
free(valor);
valor = NULL;
free(c);
c = NULL;
free(vector);
vector = NULL;
```

Llista basada en array (1/5)

- Les llistes són **estructures de dades dinàmiques** que permeten emmagatzemar-hi una **seqüència ordenada d'elements**, adaptant la seva **capacitat en funció del nombre d'elements que guarden**
- Una **forma d'implementar una llista és mitjançant un array**, encara que també n'hi ha d'altres (p.e., llistes enllaçades)
- Les següents operacions, com a mínim, haurien de ser possibles:
  - Crear llista (reservar memòria per a la llista i inicialitzar-la)
  - Alliberar llista (alliberar la memòria assignada quan no es necessiti més)
  - Afegir nou element (al final de la llista)
  - Obtenir element (donada la seva posició)
  - Inserir nou element (en una posició vàlida)
  - Eliminar element (donada la seva posició)
  - Obtenir nombre d'elements emmagatzemats

Llista basada en array (2/5)

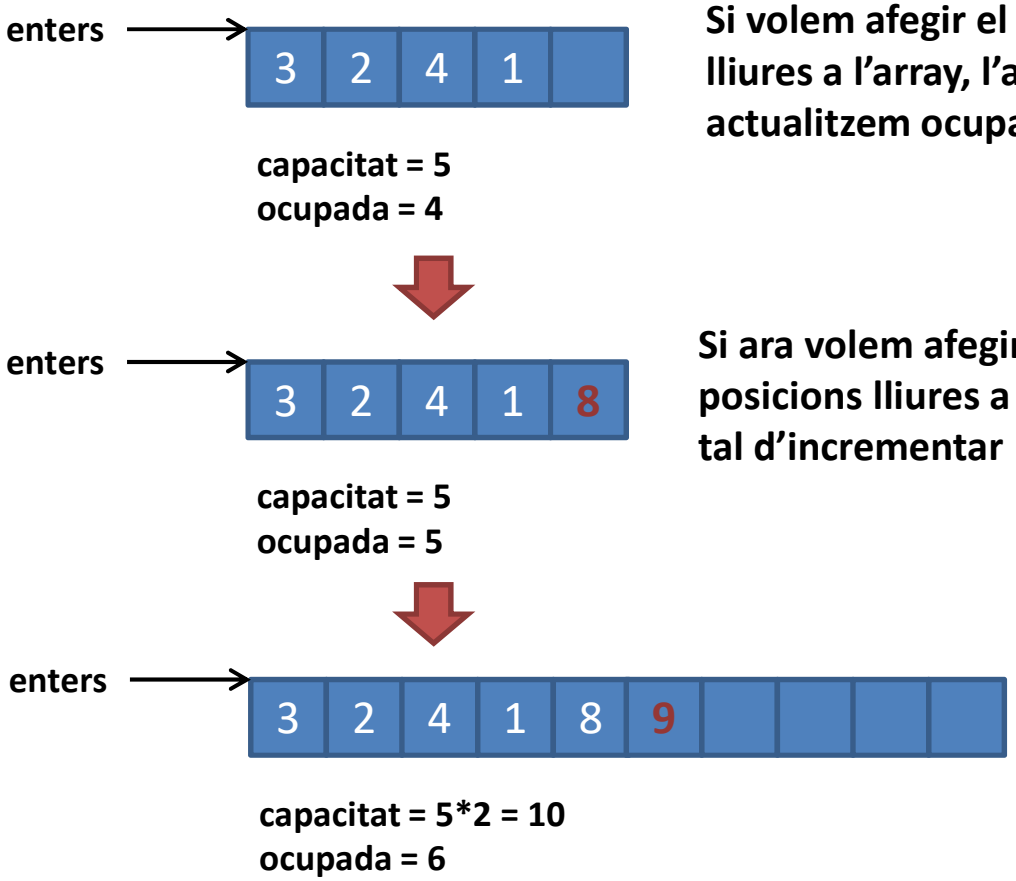
- Per a una llista de valors enters basada en array, podem definir la següent estructura:

```
typedef struct{
        int capacitat;
        int ocupada;
        int *enters;    /* Array intern de valors enters */
}t_array_list;
```

- El camp `capacitat` indica la capacitat total (mida) de l'array intern; `ocupada` indica el nombre d'elements desats actualment
- Treballant amb memòria dinàmica, representem l'array intern com a punter (`enters`), el tipus de variable que retornen les funcions `malloc()`, `calloc()` i `realloc()` i que espera `free()`

Llista basada en array (3/5)

- Funcionalitat esperada durant l'operació d'afegir nou element:



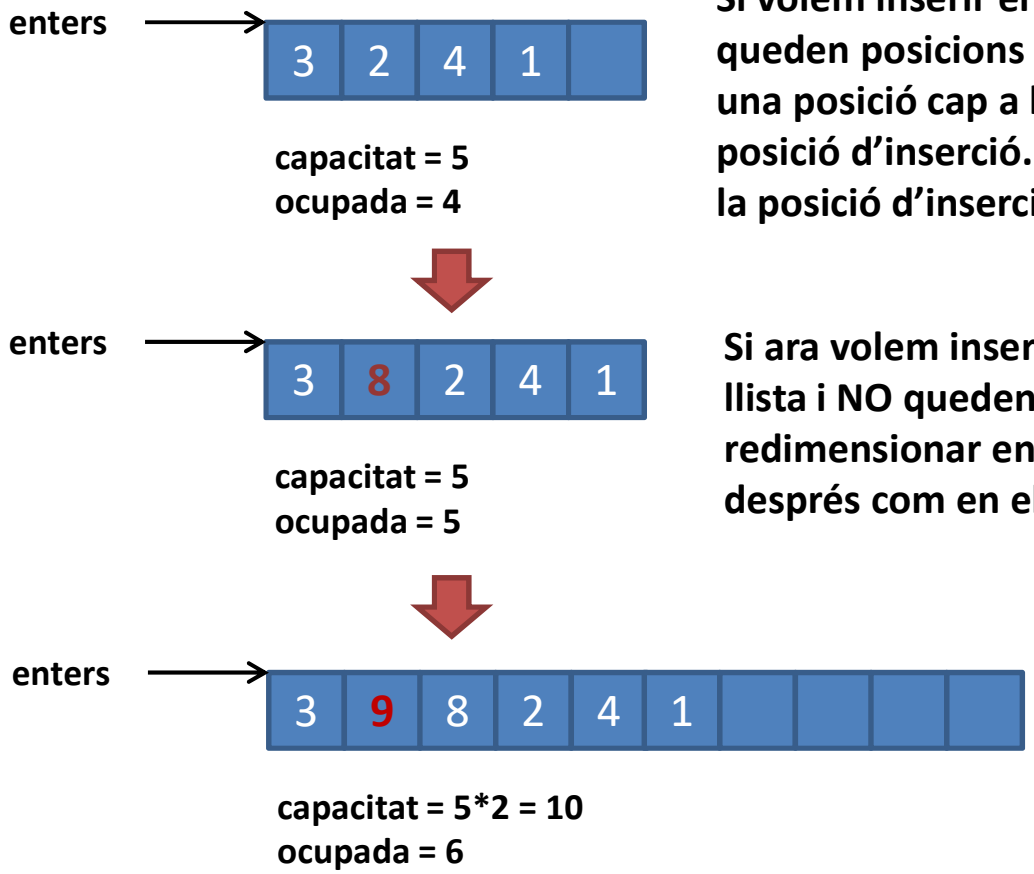
Si volem afegir el nou valor enter 8 i queden posicions lliures a l'array, l'afegim a la primera posició lliure i actualitzem ocupada = ocupada + 1

Si ara volem afegir el nou valor enter 9 i NO queden posicions lliures a l'array, caldrà redimensionar l'array per tal d'incrementar la seva capacitat abans d'afegir-lo!

Una estratègia de redimensionament típica és multiplicar x2 la capacitat

Llista basada en array (4/5)

- Funcionalitat esperada durant l'operació d'inserir nou element:

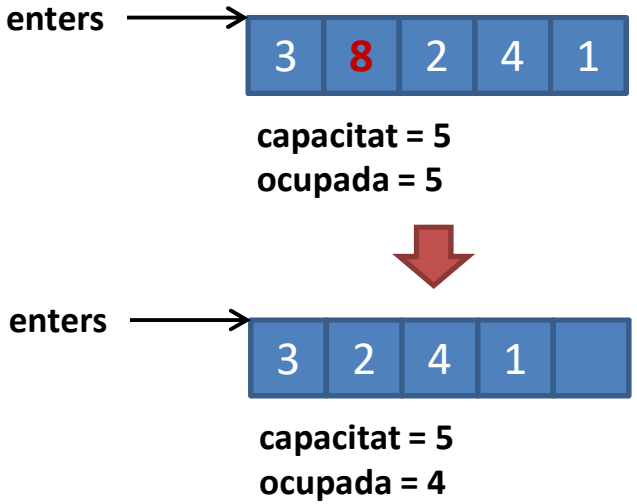


Si volem inserir el nou enter 8 a la posició 1 de la llista i queden posicions lliures a l'array, haurem de desplaçar una posició cap a la dreta tots els elements des de la posició d'inserció. Seguidament, afegim el nou element a la posició d'inserció i actualitzem ocupada = ocupada + 1

Si ara volem inserir el nou enter 9 a la posició 1 de la llista i NO queden posicions lliures a l'array, haurem de redimensionar en primer lloc l'array, per tal de procedir després com en el cas anterior

Llista basada en array (5/5)

- La funcionalitat esperada durant l'operació d'eliminar un element de la llista és semblant a la d'inserir:
  - Els elements des de la posició següent a la d'eliminació fins al final caldrà desplaçar-los tots una posició cap a l'esquerra
  - Si `ocupada` disminueix substancialment vers `capacitat`, caldrà redimensionar l'array. Per exemple, si `ocupada < capacitat/4` => `capacitat = capacitat/2`



Si volem eliminar l'element ubicat a la posició 1 de la llista, haurem de desplaçar tots els elements des de la posició 2 fins al final, una posició cap a l'esquerra

Si ocupada < capacitat/4, procediríem a redimensionar l'array, reduint-ne la seva capacitat a la meitat

Exercicis

- **<u>EXERCICI 3.1</u>**: Comença incloent la definició de l'struct `t_array_list` mostrat anteriorment al fitxer de capçalera `array_list.h`. Inclou també la definició de les següents capçaleres de funcions:

```
t_array_list * crear_llista();
void alliberar_llista(t_array_list *llista);
int add(t_array_list *llista, int valor);
void mostrar_llista(t_array_list *llista);
int inserir(t_array_list *llista, int index, int valor);
int eliminar(t_array_list *llista, int index);
```

- **<u>EXERCICI 3.2</u>**: Implementa al fitxer de codi font `array_list.c` la funció `t_array_list * crear_llista();` aquesta funció ha de reservar memòria dinàmica per a una nova llista amb capacitat igual a 5, retornant un punter a aquesta. Si succeeix qualsevol problema durant la reserva de memòria dinàmica, la funció retorna NULL.

Exercicis

- **<u>EXERCICI 3.3</u>**: Implementa al mateix fitxer la funció `void alliberar_llista t_array_list *llista);` aquesta funció ha d'alliberar tot bloc de memòria dinàmica reservat per a la llista apuntada per `llista`.
- **<u>EXERCICI 3.4</u>**: Implementa al mateix fitxer la funció `int add(t_array_list *llista, int valor);` aquesta funció ha d'afegir el valor enter `valor` al final de la llista apuntada per `llista`. La funció retorna 0 si l'enter s'ha pogut afegir o -1 si succeeix un problema de reserva de memòria dinàmica.
- **<u>EXERCICI 3.5</u>**: Implementa al mateix fitxer la funció `void mostrar_llista(t_array_list *llista)`; aquesta funció ha de mostrar per pantalla el contingut de la llista apuntada per `llista`.

Exercicis

- **<u>EXERCICI 3.6</u>**: Implementa al mateix fitxer la funció `int inserir(t_array_list *llista, int index, int valor);` aquesta funció ha d'inserir l'enter `valor` a la posició `index` de la llista apuntada per `llista`. Si cal, es realitzarà un redimensionat del seu array intern, multiplicant x2 la seva capacitat. Si ha ocorregut un problema durant la reserva de memòria, la funció retorna -1; si la posició `index` és invàlida, la funció retorna -2; en cas contrari retorna 0.
- **<u>EXERCICI 3.7</u>**: Implementa al mateix fitxer la funció `int eliminar(t_array_list *llista, int index);` aquesta funció ha d'eliminar l'enter ubicat a la posició `index` de la llista apuntada per `llista`. Un cop eliminat, si la seva capacitat ocupada és inferior a 1/4 de la seva capacitat total, es realitzarà un redimensionat, dividint la seva capacitat /2. Si ha ocurregut un problema durant la reserva de memòria, la funció retorna -1; si la posició `index` és invàlida, retorna -2; sinó retorna 0.

Exercicis

- **<u>EXERCICI 3.8</u>** : Declara al fitxer de capçalera `array_list.h` i implementa a `array_list.c` la funció `void ordenar_llista(t_array_list *llista);` aquesta funció ha d'ordenar de forma creixent els valors enters desats a la llista apuntada per llista. Implementa l'algorisme d'ordenació per inserció amb aquest propòsit.
- **<u>EXERCICI 3.9</u>** : Declara al fitxer de capçalera `array_list.h` i implementa a `array_list.c` la funció `t_array_list * clonar_llista(t_array_list *llista);` aquesta funció ha de reservar un bloc de memòria dinàmica de mida idèntica al que ocupa la llista apuntada per `llista`. Després ha d'inicialitzar-lo de forma que s'hi desi una nova llista idèntica a l'apuntada per `llista`, per acabar retornant un punter a aquest. Si succeeix qualsevol problema durant la reserva de memòria dinàmica, la funció retorna NULL.